BEGINNER'S HANDBOOK

Git on Windows

What version control is, how Git actually works, installing it on a Windows desktop, and the commands you'll use every day.

- Installation
- Core concepts
- Everyday commands
- Branching & GitHub
- Cheat sheet

Prepared for complete beginners

October 2026

Contents

1 What is Git?

2 Why Developers Use It

3 How Git Works

4 Git vs. GitHub

5 Installing Git on Windows

6 First-Time Setup

7 Basic Commands

8 Your First Repository — Walkthrough

9 Working with Branches

10 Working with a Remote (GitHub)

11 Undoing Things

12 Common Beginner Mistakes

13 Quick Reference Cheat Sheet

14 Where to Learn More

1. What is Git?

Git is a version control system — a tool that records every change made to a set of files over time, so you can look back at any previous state, see who changed what and why, and work on changes in parallel without overwriting each other.

Think of the way most people handle file versions today:

```
report.docx
report_v2.docx
report_v2_FINAL.docx
report_v2_FINAL_actually_final.docx
report_FINAL_Vivek_edits.docx
```

This "works" until you need to know what actually changed between v2 and FINAL, or two people edit at once, or you need the version from three weeks ago. Git replaces that entire mess with **one file** plus a complete, searchable history of every version it has ever had.

Git was created in 2005 by Linus Torvalds (the creator of Linux) because he needed something fast and reliable to manage the Linux kernel's source code. It is free, open source, and is now the standard version control tool used by effectively the entire software industry.

Key characteristic: Git is distributed. Every person who clones a project gets the *entire* history on their own machine, not just the latest copy. You can commit, browse history, create branches, and compare versions with no network connection at all. This is why Git feels instant compared to older centralised tools.

What Git is good for

- Source code of any kind (the main use case)
- Configuration files, infrastructure-as-code, scripts
- Documentation, Markdown, LaTeX, plain-text writing
- Anything where "what changed, when, and why" matters

What Git is *not* good for

- Large binary files (videos, big images, compiled installers) — Git stores each version in full, so the repo bloats fast. Use Git LFS or object storage for these.
- Storing secrets — passwords, API keys, and certificates do not belong in a repository, because history is permanent and shared.

2. Why Developers Use It

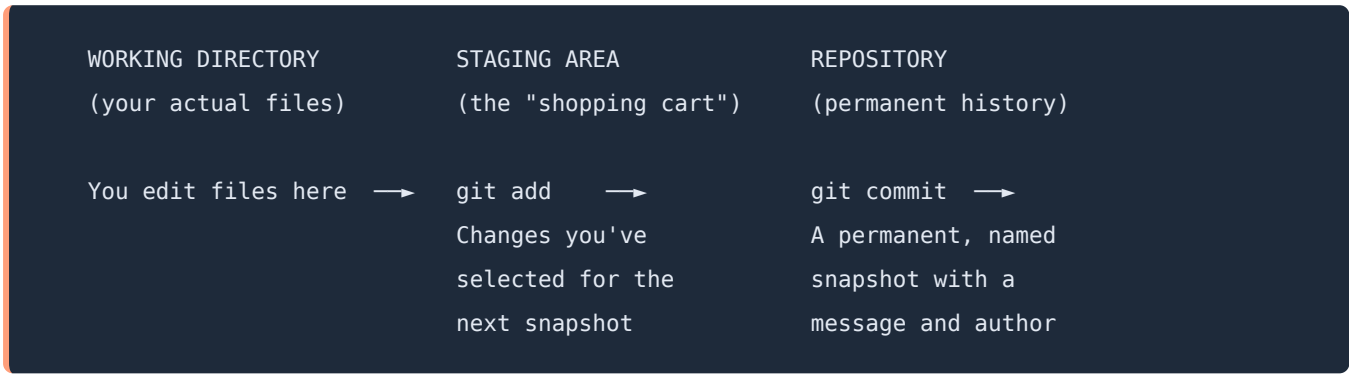
Problem	How Git solves it
"I broke something and I don't know what"	Every change is a snapshot you can inspect or revert to
"Who wrote this line and why?"	`git blame` and commit messages give you the author and the reasoning
"Two of us edited the same file"	Git merges both sets of changes, and flags only the genuinely conflicting lines
"I want to try something risky"	Branches let you experiment in isolation and throw the attempt away for free
"Where is the authoritative copy?"	A remote (GitHub, GitLab, Bitbucket, Azure DevOps) is the shared source of truth
"How do we review changes before they ship?"	Pull requests build on Git branches

3. How Git Works

This is the single most important section for a beginner. Once the mental model clicks, the commands stop feeling arbitrary.

The three areas

Git moves your work through three places on your own computer:



1. **Working directory** — the folder on disk where you edit files normally. Git watches it but doesn't touch it.
2. **Staging area** (also called the *index*) — a holding area where you choose exactly which changes go into the next snapshot. This is Git's distinctive feature: you can edit ten files and commit only three, so each commit stays focused and meaningful.
3. **Repository** — the `.git` folder, where every committed snapshot lives forever.

A fourth area appears once you work with others:

4. **Remote** — a copy of the repository hosted on a server (GitHub, for example). You `push` your commits to it and `pull` other people's commits from it.

What a commit actually is

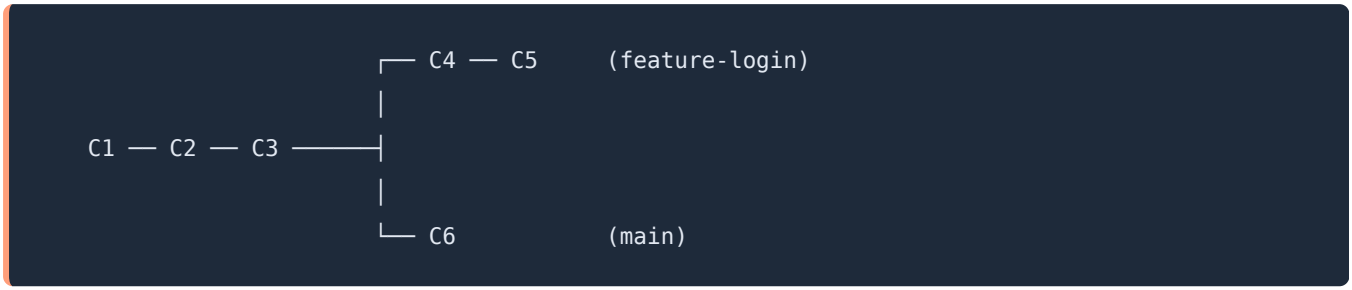
A commit is a **complete snapshot** of your project at one moment, plus metadata:

- A unique ID — a 40-character SHA-1 hash like `a3f7c9e2b1...` (you usually only type the first 7 characters)
- Author name and email
- Timestamp
- A message describing *why* the change was made
- A pointer to the commit that came before it

Because each commit points to its parent, the full history forms a chain. That chain is why you can travel back to any point, and why history is tamper-evident — change one old commit and every ID after it changes too.

Branches

A branch is simply **a movable pointer to a commit**. That's all. Creating a branch costs nothing — Git writes a file containing one commit ID. This is why branching in Git is cheap and constant, while in older systems it was a big, scary operation.



You work on `feature-login`, and when it's ready you **merge** it back into `main`. If both branches changed the same lines of the same file, Git asks you to resolve the **conflict** manually — it will never silently guess.

HEAD

`HEAD` is a pointer to "where you are right now" — normally the tip of the branch you have checked out. When you `git checkout` a different branch, Git moves HEAD and swaps the files in your working directory to match.

4. Git vs. GitHub

Beginners mix these up constantly, so to be explicit:

	Git	GitHub
What it is	A program on your computer	A website / hosting service
Who made it	Linus Torvalds, 2005	GitHub Inc. (owned by Microsoft)
Needs internet?	No	Yes
Cost	Free, open source	Free tier + paid plans
Purpose	Tracks versions	Stores repos online, adds collaboration tools (pull requests, issues, CI, code review)

Analogy: Git is like the camera; GitHub is like Instagram. You can use the camera without ever posting anything. Alternatives to GitHub include GitLab, Bitbucket, and Azure DevOps — they all speak the same Git.

5. Installing Git on Windows

The official Windows build is **Git for Windows**. It bundles the Git command-line tools, **Git Bash** (a Unix-style shell, which is what most tutorials assume), and **Git GUI**.

At the time of writing, the current release is in the **2.56.x** series, but always take whatever the site offers — it will be newer.

Method A — The official installer (recommended for beginners)

Step 1 — Download

Go to **https://git-scm.com/download/win**. The download starts automatically for 64-bit Windows, which is what virtually every modern PC runs. You'll get a file named something like `Git-2.56.0-64-bit.exe`.

Step 2 — Run the installer

Double-click the `.exe`. Click **Yes** on the Windows User Account Control prompt.

Step 3 — Work through the setup screens

There are roughly a dozen screens. The defaults are sensible — if you are unsure, **press Next**. The ones worth a second of thought:

Screen	Recommendation
Destination location	Leave as `C:\Program Files\Git`
Select components	Keep the defaults. Tick **"Add a Git Bash Profile to Windows Terminal"** if offered — it's convenient
Default editor used by Git	Default is Vim, which is confusing if you've never used it. **Change this to Notepad++ or Visual Studio Code** if you have one installed; otherwise pick Notepad
Initial branch name	Choose **"Override… `main`"**. `main` is the modern convention; the old default was `master`
Adjusting your PATH	Keep **"Git from the command line and also from 3rd-party software"** (the middle, recommended option). This makes `git` work in Command Prompt and PowerShell too, not just Git Bash
SSH executable	Use bundled OpenSSH
HTTPS transport backend	Use the OpenSSL library (default). Pick the Windows Secure Channel option only if you're on a corporate network with its own certificate authority
Line ending conversions	Keep **"Checkout Windows-style, commit Unix-style line endings"** (default). This is the right answer on Windows
Terminal emulator	Use **MinTTY** (default) — it's a much better terminal than the old Windows console
Credential helper	Keep **Git Credential Manager**. It securely stores your GitHub login so you don't retype it
Extra options	Leave file system caching enabled

Click **Install**, wait a minute, then **Finish**.

Step 4 — Verify the installation

Open the Start menu, type `Git Bash`, and open it. Then run:

```
git --version
```

You should see something like:

```
git version 2.56.0.windows.1
```

If you see a version number, you're done. Also try the same command in PowerShell — if it works there too, your PATH is set up correctly.

Method B — Windows Package Manager (winget)

If you're comfortable with a terminal, this is a one-liner. Open PowerShell or Command Prompt:

```
winget install --id Git.Git -e --source winget
```

`winget` ships with Windows 11 and recent Windows 10 builds. Close and reopen your terminal afterwards so the PATH refreshes, then check with `git --version`.

Method C — Chocolatey

If you already use the Chocolatey package manager, run PowerShell **as Administrator**:

```
choco install git -y
```

Keeping Git updated

Git for Windows can update itself:

```
git update-git-for-windows
```

Or just re-run the latest installer over the top — it upgrades in place and keeps your settings.

Which terminal should I use?

You have three options, and all of them work:

- **Git Bash** — recommended for beginners. It gives you the Unix commands (`ls`, `cat`, `grep`, `touch`) that almost every Git tutorial on the internet assumes you have.
- **PowerShell** — fine, and already familiar if you use Windows a lot. Use `dir` instead of `ls`, `type` instead of `cat`.
- **Windows Terminal** — a modern tabbed terminal that can host Git Bash, PowerShell, and Command Prompt side by side. Install it from the Microsoft Store if you don't have it.

6. First-Time Setup

Run these once per computer. Git stamps your name and email onto every commit you make — they are not secret, and they are what shows up in the project history.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

> Use the same email address as your GitHub account, otherwise GitHub won't link your commits to your profile.

A few more settings worth doing on Windows:

```
# Make 'main' the default branch name for new repositories
git config --global init.defaultBranch main

# Handle Windows/Unix line endings correctly
git config --global core.autocrlf true

# Set your editor (example: VS Code)
git config --global core.editor "code --wait"

# Nicer, more readable output
git config --global color.ui auto
```

Check what you've set:

```
git config --list
```

These live in a plain text file at `C:\Users\<YourName>\.gitconfig` — you can open and edit it directly if you prefer.

7. Basic Commands

Starting a repository

Command	What it does
`git init`	Turn the current folder into a Git repository
`git clone <url>`	Download an existing repository (and its full history) from a remote

The everyday cycle

This is the loop you'll run dozens of times a day:

Command	What it does
`git status`	**Run this constantly.** Shows what's changed, what's staged, what branch you're on
`git add <file>`	Stage one file for the next commit
`git add .`	Stage everything that changed in the current folder and below
`git commit -m "message"`	Save the staged changes as a permanent snapshot
`git log`	Show the commit history
`git log --oneline --graph --all`	A compact visual history — far more readable
`git diff`	Show changes you've made but not yet staged
`git diff --staged`	Show changes you've staged but not yet committed

Working with remotes

Command	What it does
`git remote -v`	List the remotes this repo is connected to
`git remote add origin <url>`	Connect your local repo to a remote (`origin` is the conventional name)
`git push`	Send your commits to the remote
`git pull`	Fetch the remote's commits and merge them into your branch
`git fetch`	Download remote commits *without* merging them — safer when you want to look first

Branching

Command	What it does
`git branch`	List branches (the current one is marked with `*`)
`git branch <name>`	Create a new branch
`git switch <name>`	Switch to an existing branch
`git switch -c <name>`	Create a branch *and* switch to it in one step
`git merge <name>`	Merge the named branch into your current branch
`git branch -d <name>`	Delete a branch that has already been merged

> `git checkout` does the job of both `git switch` and `git restore`. Older tutorials use `checkout` everywhere; `switch` and `restore` are the newer, clearer commands and are what you should learn first.

Writing good commit messages

A commit message explains *why*, not *what* — the diff already shows what changed.

```
# Weak
git commit -m "fixed stuff"
git commit -m "update"

# Good
git commit -m "Fix login timeout when session cookie is missing"
git commit -m "Add input validation to the invoice date field"
```

Convention: write the summary line in the imperative mood ("Add", "Fix", "Remove" — as if completing the sentence "This commit will…"), keep it under about 50 characters, and capitalise the first word.

8. Your First Repository — Walkthrough

Open Git Bash and follow along. Every line is safe to run.

```
# 1. Create a folder and move into it
mkdir my-first-repo
cd my-first-repo

# 2. Turn it into a Git repository
git init
# → Initialized empty Git repository in .../my-first-repo/.git/

# 3. Create a file
echo "# My First Project" > README.md

# 4. See what Git thinks
git status
# → README.md shows up as an "untracked file" (Git sees it but isn't tracking it yet)

# 5. Stage it
git add README.md

# 6. Check again
git status
# → README.md is now under "Changes to be committed"

# 7. Commit it
git commit -m "Add project README"
# → [main (root-commit) a3f7c9e] Add project README

# 8. View the history
git log --oneline
# → a3f7c9e Add project README
```

Now make a change and repeat the cycle:

```
echo "This project is a Git practice exercise." >> README.md

git diff          # see exactly what changed, line by line
git add README.md
git commit -m "Describe the purpose of the project"
git log --oneline # you now have two commits
```

That edit → `add` → `commit` loop is roughly 80% of day-to-day Git.

The `.gitignore` file

Some files should never be committed — build output, dependency folders, local settings, secrets. Create a file named `.gitignore` in the root of your repo:

```
# Dependencies
node_modules/
venv/

# Build output
dist/
build/
*.exe

# Windows junk
Thumbs.db
desktop.ini

# Editor settings
.vscode/
.idea/

# Secrets — never commit these
.env
*.key
```

Git will now ignore anything matching these patterns. GitHub maintains ready-made templates per language at https://github.com/github/gitignore.

9. Working with Branches

```
# Create a branch and switch to it
git switch -c feature/add-contact-form

# ... edit files, then:
git add .
git commit -m "Add contact form with email validation"

# Go back to main
git switch main

# Bring your work in
git merge feature/add-contact-form

# Clean up
git branch -d feature/add-contact-form
```

When a merge conflict happens

If the same lines changed on both branches, Git stops and marks the file:

```
<<<<<<< HEAD
This is the version on your current branch.
=======
This is the version from the branch you're merging in.
>>>>>>> feature/add-contact-form
```

To resolve it:

1. Open the file and edit it so it reads the way you want it to.
2. Delete the `<<<<<<<`, `=======`, and `>>>>>>>` marker lines.
3. `git add <file>`
4. `git commit`

Conflicts feel alarming the first time, but they're routine — Git is asking you a question, not reporting a failure. If you want out entirely, `git merge --abort` returns everything to how it was before you started.

10. Working with a Remote (GitHub)

Pushing an existing local repo to GitHub

Create an empty repository on GitHub first (don't let it add a README), then:

```
git remote add origin https://github.com/yourusername/my-first-repo.git
git branch -M main
git push -u origin main
```

The `-u` sets the upstream, so from then on plain `git push` and `git pull` know where to go.

Cloning an existing project

```
git clone https://github.com/someone/their-project.git
cd their-project
```

Authentication on Windows

GitHub stopped accepting account passwords for Git operations in 2021. Git Credential Manager — installed by default with Git for Windows — handles this for you: the first time you push, a browser window opens, you sign in to GitHub once, and the credentials are stored securely in Windows Credential Manager. Nothing further to configure.

If you prefer SSH keys instead, generate one in Git Bash:

```
ssh-keygen -t ed25519 -C "you@example.com"
cat ~/.ssh/id_ed25519.pub
```

Then paste that public key into GitHub under **Settings → SSH and GPG keys**.

The typical daily collaboration loop

```
git pull                            # get everyone else's latest work
git switch -c feature/my-task       # branch off for your change
# ... work ...
git add .
git commit -m "Describe what you did"
git push -u origin feature/my-task    # send it up
# ... then open a Pull Request on GitHub for review
```

11. Undoing Things

Everything here is reversible, but the commands differ depending on *how far along* the change is.

Situation	Command
Discard changes to a file you haven't staged	`git restore <file>`
Unstage a file (keep the edits)	`git restore --staged <file>`
Fix the message of the most recent commit	`git commit --amend -m "Better message"`
Undo a commit, keep the changes in your working directory	`git reset --soft HEAD~1`
Undo a commit and throw the changes away	`git reset --hard HEAD~1` ⚠️ **destructive**
Reverse a commit by adding a new, opposite commit	`git revert <commit-id>` ✅ **safe for shared branches**
Shelve work temporarily	`git stash` → bring it back with `git stash pop`

> **Rule of thumb:** use `reset` on commits that only exist on your own machine, and `revert` on anything you've already pushed. Rewriting pushed history breaks it for everyone else who has pulled it.
>
> **The safety net:** `git reflog` lists everywhere HEAD has been, including commits you thought you destroyed. If you've lost work in the last 90 days, it's almost certainly still recoverable from here.

12. Common Beginner Mistakes

1. **Committing secrets.** Once an API key is in history, removing it is painful and you must assume it's compromised. Use `.gitignore` and environment variables from day one.
2. **Giant commits.** "Changes from today" with 40 files tells nobody anything. Commit one logical change at a time.
3. **Never running `git status`.** It's free and it tells you exactly where you stand. Run it before and after everything.
4. **Working directly on `main`.** Branch for anything non-trivial.
5. **Committing `node_modules/` or `venv/`.** Thousands of files, all reinstallable. `.gitignore` them.
6. **Forgetting to pull before starting work.** You'll create conflicts that didn't need to exist.
7. **Panicking at a conflict.** Nothing is broken. Edit the file, remove the markers, add, commit.
8. **Using `git reset --hard` casually.** It's the one genuinely destructive command on this page. Pause before you run it.

13. Quick Reference Cheat Sheet

```
# ── SETUP ──────────────────────────────────────────────
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list

# ── START ──────────────────────────────────────────────
git init                          # new repo here
git clone <url>                   # copy an existing repo

# ── DAILY CYCLE ────────────────────────────────────────
git status                        # where am I?
git add <file>                    # stage one file
git add .                         # stage everything
git commit -m "message"           # snapshot it
git log --oneline --graph --all # see the history

# ── INSPECT ────────────────────────────────────────────
git diff                          # unstaged changes
git diff --staged                 # staged changes
git show <commit>                 # one commit in detail
git blame <file>                  # who changed each line

# ── BRANCHES ───────────────────────────────────────────
git branch                        # list
git switch -c <name>              # create + switch
git switch <name>                 # switch
git merge <name>                  # merge into current branch
git branch -d <name>              # delete merged branch

# ── REMOTES ────────────────────────────────────────────
git remote -v                     # list remotes
git remote add origin <url>     # connect
git push -u origin main         # first push
git push                          # subsequent pushes
git pull                          # fetch + merge
git fetch                         # fetch only

# ── UNDO ───────────────────────────────────────────────
git restore <file>              # discard unstaged edits
git restore --staged <file>     # unstage
```

```
git commit --amend              # fix last commit
git revert <commit>             # safe undo (new commit)
git reset --hard HEAD~1         # ⚠️ destructive undo
git stash / git stash pop       # shelve / unshelve
git reflog                      # find "lost" commits
```

14. Where to Learn More

- **Official Git documentation** — https://git-scm.com/doc
- **Pro Git book** (free, and genuinely the best resource) — https://git-scm.com/book/en/v2
- **Git for Windows releases** — https://github.com/git-for-windows/git/releases
- **Learn Git Branching** (interactive visual tutorial) — https://learngitbranching.js.org
- **GitHub Docs** — https://docs.github.com/en/get-started
- **Oh Shit, Git!?!** (fixes for common messes) — https://ohshitgit.com

GUI clients, if you'd rather not use the terminal

- **GitHub Desktop** — simplest, free: https://desktop.github.com
- **VS Code** — excellent Git support built in, no extension needed
- **Sourcetree** — free, more powerful: https://www.sourcetreeapp.com
- **GitKraken** — polished, free for public repos

Learn the command line anyway. Every GUI is a wrapper around these same commands, and when something goes wrong, the terminal is where you fix it.

Happy committing.